Lalitha Barri

sailalithabarri@gmail.com| New York, NY (Open to relocation) linkedin.com/in/lalithabarri | github.com/LalithaBarri

Education

Stony Brook University – SUNY Aug 2024 – May 2026
M.S. in Computer Science; GPA: 3.53/4.0 Stony Brook, NY

Coursework: Distributed Systems, Operating Systems, Database Systems, Machine Learning

Vellore Institute of Technology Aug 2020 – May 2024
B.Tech. in Computer Science and Engineering; GPA: 3.5/4.0 India

Coursework: Data Structures & Algorithms, Computer Networks, Object-Oriented Programming, Software Engineering

Professional Experience

CEAS – Stony Brook University Jan 2026 – Apr 2026
Software Engineer Intern Stony Brook, NY

- Built a React/FastAPI researcher-search application across university departments, replacing manual lookup with ranked results over **233 researcher profiles** at **p95 search latency below 300 ms**.
- Improved top-three retrieval relevance (**Hit Rate@3: 42% with BM25 to 86%**) on 50 logged user queries using pgvector retrieval and one bounded LangGraph reformulation retry in a profile-grounded RAG workflow; added pytest checks to detect retrieval regressions.
- Traced 24 duplicate profiles to concurrent ingestion retries; implemented transactional PostgreSQL upserts and namespace-scoped identity matching, preventing repeat inserts in reproduced failure cases.
- Replaced incompatible ChromaDB storage with PostgreSQL/pgvector to unblock deployment; added REST contract checks in GitHub Actions, request tracing, and Docker packaging for AWS deployment.

Brookhaven National Laboratory Jan 2025 – Dec 2025
Graduate Research Assistant Upton, NY

- Accelerated CNN/ViT evaluation for brain tumor classification by **3**× over the uncached pipeline on 3,000 MRI slices by profiling repeated feature extraction and caching reusable vectors in PyTorch.
- Consolidated duplicated scripts for **15 PyTorch variants** into shared training, checkpointing, and evaluation modules; tracked configurations, metrics, and artifacts in MLflow for repeatable comparisons.
- Investigated scanner-specific background cues with Grad-CAM and compared preprocessing variants in MLflow; selected intensity normalization over background masking based on cross-scanner evaluation.
- Containerized the model-pipeline runtime with Docker and parallelized pytest in GitHub Actions, returning accuracy and calibration checks in under 10 minutes before merging pipeline changes.

SpiraLinks Jan 2024 – Aug 2024
Software Development Engineer India

- Reduced reporting-endpoint **p95 latency by 56% to below 100 ms** by fixing N+1 queries, adding PostgreSQL composite indexes, and caching repeated reads in Redis.
- Implemented REST contracts during a team migration from a Java monolith to six Spring Boot services; added JUnit regressions and participated in code reviews to check interface compatibility.
- Validated OAuth 2.0 JWT access tokens and enforced server-side role-based access control; added JUnit tests confirming rejection of invalid tokens and requests with insufficient permissions.
- Diagnosed repeated container restarts during database delays; separated Kubernetes liveness and readiness checks so database availability affected traffic routing without triggering unnecessary application restarts.

Technical Skills

Languages: Python, Java, C++, TypeScript, JavaScript, SQL, HTML/CSS
Frameworks & APIs: React, FastAPI, Spring Boot, REST APIs, gRPC, Temporal
Databases & Retrieval: PostgreSQL, pgvector, Redis, DynamoDB, vector search
Cloud & Delivery: AWS, Docker, Kubernetes, Terraform, Git, GitHub Actions, Linux
AI/ML: PyTorch, MLflow, LangGraph, RAG, OpenAI API, LLM/agent evaluation
Testing & Observability: pytest, JUnit, Playwright, CloudWatch, OpenTelemetry

Projects

LegacyBank Capability Compiler May 2026 – Sept 2026
Tech Stack: Python, Playwright, Meta Llama 3 70B

- Converted LLM-discovered banking workflows into typed, versioned capabilities that Playwright could replay with zero model calls; parameterized replay inputs and selected discovery actions from observed browser state.
- Enforced deny-by-default action checks and human approval for irreversible transfers; handed control to a human in the same browser session and verified postconditions before resuming automation.
- Validated discovery, replay, and business postconditions with **888 tests, including 36 browser tests**, covering ambiguous targets, policy denials, and retry blocking after unknown irreversible transaction outcomes.

Waypoint – Collaborative AI Travel Planner Jan 2026 – Apr 2026
Tech Stack: TypeScript, Temporal, PostgreSQL, OpenTelemetry, OpenAI API

- Built itinerary planning around LLM activity suggestions and a deterministic scheduler enforcing time-window constraints, reducing **median travel time by 18%** against a greedy baseline in the evaluation workload.
- Built Temporal/PostgreSQL workflows for itinerary planning and tested 100 worker-failure scenarios; used optimistic concurrency control to detect stale edits across 100 concurrent-edit pairs.
- Reduced OpenAI **API spend by 24%** versus unconstrained prompting through bounded tool-call workflows and caching; **96%** of evaluated plans passed time-window and geographic-routing checks, with OpenTelemetry tracing for failure diagnosis.

Streamline – Event-Driven Order Processing Jul 2025 – Dec 2025
Tech Stack: AWS Lambda, API Gateway, SNS/SQS, DynamoDB, Terraform, CloudWatch

- Decoupled order admission from processing with API Gateway, Lambda, and SNS/SQS after a synchronous path degraded under bursts, allowing accepted requests to queue independently of worker execution.
- Recorded zero duplicate order writes in a **150K-event test** at an offered load of **250 requests/sec** with duplicate delivery; rejected repeated operations using DynamoDB conditional writes keyed to the business operation.
- Bounded retries and routed exhausted messages to an SQS dead-letter queue; provisioned infrastructure with Terraform and monitored queue backlog and Lambda failures through CloudWatch.

Quorum – Distributed Key-Value Store Jan 2025 – Apr 2025
Tech Stack: Python, Raft, gRPC, AWS Graviton (ARM64)

- Implemented Raft leader election and log replication behind a gRPC key-value API, with snapshots and restart recovery to restore replicated state after node restarts.
- Traced recurring split votes during simulated partitions to synchronized election deadlines; randomized election timeouts to reduce collisions after leader loss.
- Checked histories for linearizability against a sequential model under leader failure, log divergence, and partitions; separately measured **3K+ completed operations/sec** on a five-node AWS Graviton cluster with an 80/20 read/write workload.

LSM-Tree Key-Value Storage Engine Aug 2024 – Dec 2024
Tech Stack: C++, SSTables, Bloom Filters, MurmurHash, CRC32

- Built a persistent C++ key-value engine with MemTables, append-only logging, and SSTables; implemented background compaction to merge segments and reclaim space occupied by obsolete entries.
- Added MurmurHash-based Bloom filters to skip SSTables for definitely absent keys, avoiding unnecessary disk reads on negative lookups.
- Diagnosed partial-log recovery failures through SIGKILL testing; added length-prefixed records, CRC32 validation, and handling for incomplete trailing records to recover valid preceding data after process crashes.

Publication

Bird Species Recognition using Deep Learning – IEEE AISP 2023.